

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

Playwright Locator Strategy Cheat Sheet

Choose robust, user-facing locators that do not break.

Introduction

Locators are where automation succeeds or fails. This cheat sheet shows the recommended Playwright locators in priority order, so your tests are stable and readable.

Locator Priority (best first)

Locator	Use when
<code>getByRole</code>	Almost always — buttons, links, headings, inputs by role + name
<code>getByLabel</code>	Form fields with a label
<code>getByPlaceholder</code>	Inputs identified by placeholder
<code>getByText</code>	Non-interactive text content
<code>getByTestId</code>	When nothing user-facing is stable (data-testid)
CSS / XPath	Last resort only

Worked Example

- `page.getByRole('button', { name: 'Sign in' })`
- `page.getByLabel('Email address')`
- `page.getByRole('heading', { name: 'Dashboard' })`
- `page.getByTestId('cart-total')`

Why User-Facing Locators Win

- They survive CSS/markup refactors.
- They double as accessibility checks.
- They read like the user's intent.

Practical Exercise

Take a page and locate five elements using only `getByRole`/`getByLabel`/`getByText`. Only fall back to `getByTestId` if you truly cannot.

Best Practices

- Prefer role + accessible name.
- Add `data-testid` for genuinely ambiguous elements.
- Avoid brittle CSS chains and indexes.

Common Mistakes

- Long CSS selectors tied to layout.
- Using `nth-child` / indexes that shift.
- XPath everywhere "because it works".

INSIDE STLC PRO TIP

A good rule: locate elements the way a user would describe them. If you would say "the Sign in button", use `getByRole('button', { name: 'Sign in' })`.